

Algoritmo Exato para problema NP-Difícil: Nonogramas

Gabriel Henrique Campos Medeiros

Novembro 2025

1 Introdução

Nonogramas é um quebra-cabeça que consiste em colorir células em uma grade $N \times M$ para revelar uma imagem, com base em restrições numéricas fornecidas para cada linha e coluna, que indicam o tamanho e ordem dos blocos de célula que serão preenchidas.

Um nonograma pode ser enxergado, portanto, como um problema de satisfação de restrições, o que acaba por classificar ele como um problema NP-Completo, tornando a busca pela solução um problema intratável no pior caso, demandando a implementação de técnicas robustas para a solução.

Este trabalho apresentará uma solução baseada na técnica de permutações de blocos em junção de um algoritmo de backtracking. A técnica das permutações é utilizada para resolver o máximo de células através da certeza lógica, minimizando o espaço de busca. A fase do backtracking é empregada para resolver as ambiguidades restantes.

O relatório detalhará a complexidade do problema, a lógica da técnica aplicada e também as provas de correção.

2 O problema: Nonogramas

O nonograma é um quebra-cabeça de lógica apresentado em uma grade de $N \times M$ células. O objetivo é determinar quais células devem ser preenchidas (normalmente de preto) e quais devem ser deixadas vazias (normalmente brancas), de modo que o preenchimento em cada linha e coluna corresponda às restrições fornecidas.

As restrições são dadas como sequências de números inteiros positivos. Por exemplo, $(3,1,2)$ para uma linha indica que existem, em ordem, um bloco de três células pretas, seguido por pelo menos uma célula branca, depois um bloco de uma célula preta, seguido por pelo menos uma célula branca, e finalmente um bloco de duas células pretas (seguido por pelo menos uma ou nenhuma células brancas).

O problema de decidir se uma instância de nonograma tem uma solução é classificado como NP-Completo. Essa classificação foi comprovada por Batenburg [1]. Outra prova da NP-completude pode ser encontrada na seguinte referência [7]. Esta classificação NP-Completa tem implicações diretas e profundas na escalabilidade dos solvers. Ela estabelece que, para instâncias de Nonogramas suficientemente grandes e complexos, qualquer algoritmo de busca exaustiva (como o backtracking) estará sujeito a um tempo de execução exponencial no pior caso. Consequentemente, o desenvolvimento de solvers de alto desempenho não pode se basear unicamente na força bruta, mas deve focar em estratégias que maximizem o raciocínio lógico em tempo polinomial antes de recorrer à busca exponencial, deslocando o gargalo de desempenho para a eficácia do pré-processamento lógico.

O nonograma se encaixa naturalmente na estrutura de um problema de satisfação de restrições (CSP). Nessa modelagem, as variáveis são as células da matriz, cada uma com um domínio discreto de valores [3]. A eficácia de um solver CSP reside na aplicação de inferências lógicas para reduzir o domínio de valores das células, minimizando a necessidade de escolhas arbitrárias e, consequentemente, reduzindo o fator de ramificação do backtracking.

A espinha dorsal de qualquer solver de Nonograma eficiente é a sua capacidade de derivar informação lógica em tempo polinomial. O estado da arte reconhece que o desempenho é maximizado quando a busca exponencial é adiada e o espaço de busca é drasticamente reduzido através de inferências lógicas. Algumas das técnicas heurísticas clássicas para resolver o nonograma são: Overlap, Forcing e a técnica conjunta de Glue, Joining e Splitting [2].

A resolução exata e eficiente de uma única restrição é um subproblema 1D crucial. A programação dinâmica é o método padrão para este propósito. Historicamente, os melhores métodos de line-solving baseados em programação dinâmica apresentavam uma complexidade de tempo de pior caso $O(kl^2)$, onde l é o comprimento da linha ou coluna e k é o número de inteiros nas pistas [6]. O estado da arte evoluiu significativamente. Abordagens mais recentes propuseram um método rápido de programação dinâmica para line-solving cuja complexidade no pior caso é de apenas $O(kl)$ [6].

Para além destas técnicas mais focadas no lado exato, também existem abordagens no estado da arte focadas em busca exaustiva, meta-heurísticas e inteligência artificial, porém este não é o atual foco desse relatório.

3 Descrição da técnica escolhida

A técnica central escolhida para o desenvolvimento do algoritmo exato é a Propagação de Restrições baseada em Permutações (Full Scan).

A Heurística de Permutação atua em uma única linha ou coluna por vez e tem como objetivo determinar o estado de cada célula (Preto ou Branco) com certeza absoluta, dadas as restrições numéricas daquela linha e o estado atual da grade.

A ideia desta técnica é que dada as restrições de uma determinada linha ou

coluna, o algoritmo deve gerar todas as sequências possíveis de preenchimento de blocos que satisfaçam as restrições de tamanho e espaçamento e seja também consistente com as células que já foram marcadas no tabuleiro. A recursão é interrompida caso e um determinado ramo de soluções pode ser podado caso a tentativa de encaixe de um bloco:

- Caia sobre uma célula já marcada como branco.
- Deixar uma célula preta já marcada sem ser coberta pelo bloco.
- Gastar mais espaço do que o restante da linha permite, deixando a sequência de blocos incompleta.

Após gerar todas as permutações, o algoritmo realiza a superposição de todos os resultados.

4 Implementação e análise

O resolvidor de nonograma foi implementado em C++. A estrutura central dos dados é o struct `Nonograma`, que engloba os componentes necessários para representar e resolver qualquer instância do puzzle:

```
1      using Restricoes = vector<int>;
2
3  struct Nonograma{
4      int linhas = 0;
5      int colunas = 0;
6
7      vector<Restricoes> col_restr;
8      vector<Restricoes> lin_restr;
9
10     vector<vector<int>> grid;
11 };
```

Uma instância de nonograma é recebida através de um formato estruturado que alimenta o struct `nonograma`. A função utilizada se chama "carregar nonograma" e lê um arquivo txt fornecido pelo usuário. Este arquivo txt deve ser composto por uma lista de números separados por espaço, onde os dois primeiros números devem se referir ao tamanho do tabuleiro, o número seguinte `N` representa a quantidade de dicas na primeira coluna e os `N` números seguinte são o tamanho dos blocos da respectiva coluna. O resto das dicas para o resto das colunas e linhas segue a mesma lógica. Vale ressaltar que de início o tabuleiro é preenchido com o valor 0, representando que não foi preenchido nem com preto nem branco.

Uma vez carregado uma instância de problema do nonograma, podemos aplicar a função "solve nonogram". Esta função é dividida em duas partes: A parte determinística onde aplicamos a técnica descrita na seção anterior, e uma parte de busca exaustiva para quando a técnica anterior não é o suficiente.

```

1  bool changed = true;
2  int iterations = 0;
3  while (changed && iterations < 100) {
4      changed = false;
5      iterations++;
6
7      for (int i = 0; i < instancia.linhas; ++i) {
8          if (aplicar_permutacoes(instancia.grid[i],
9              instancia.lin_restr[i]))
10             changed = true;
11     }
12     for (int j = 0; j < instancia.colunas; ++j) {
13         vector<int> col;
14         if (aplicar_permutacoes(col,
15             instancia.col_restr[j])) {
16             changed = true;
17         }
18     }
19 }

```

O trecho acima representa o núcleo da fase heurística, onde a heurística de permutação é aplicada iterativamente até que o tabuleiro se estabilize, ou seja, mais uma aplicação da heurística não muda o tabuleiro. O segmento aplica a heurística a partir da função "Aplicar permutações", que alternadamente é aplicada em todas as linhas e colunas até que não haja mais nenhuma célula resolvida com certeza lógica.

O loop garante que as consequências de uma marcação em uma linha sejam propagadas para as colunas, e as consequências disso voltem para as linhas, em um processo iterativo, até que a grade atinja um ponto fixo determinístico, onde nenhuma nova célula pode ser resolvida com certeza lógica. Neste ponto, o `changed` permanece `false`, e o algoritmo pode prosseguir para o backtracking.

```

1  bool aplicar_permutacoes(vector<int>& cells,
2  const Restricoes& restr) {
3      if (restr.empty()) {
4          return true;
5      }
6
7      int tamanho_linha = cells.size();
8      vector<int> contagem_preto(tamanho_linha, 0);
9      vector<int> contagem_branco(tamanho_linha, 0);
10     int total_permutacoes_validas = 0;
11
12     gerar_permutacao(
13         0,
14         0,
15         cells,
16         restr,
17         contagem_preto,

```

```

18         contagem_branco,
19         total_permutacoes_validas
20     );
21
22     if (total_permutacoes_validas == 0) {
23         return false;
24     }
25
26     bool mudou = false;
27
28     if (total_permutacoes_validas > 0) {
29         for (int k = 0; k < tamanho_linha; ++k) {
30             if (cells[k] == 0) {
31                 if (contagem_preto[k] ==
32                     total_permutacoes_validas) {
33                     cells[k] = 1;
34                     mudou = true;
35                 }
36                 else if (contagem_branco[k] ==
37                     total_permutacoes_validas) {
38                     cells[k] = -1;
39                     mudou = true;
40                 }
41             }
42         }
43     }
44
45     return true;
46 }

```

Vamos falar então da função "aplicar permutacoes". Essa função recebe uma linha (ou coluna) e as restrições dela. A linha 3 na tabela acima representa o caso base, caso a linha ou coluna não possuir restrições, a função retorna true. O próximo passo relevante ocorre na linha 12, onde chamamos a função "gerar permutacao".

```

1  void gerar_permutacao(
2      int indice_bloco,
3      int posicao_atual,
4      vector<int> current_cells,
5      const Restricoes& restr,
6      vector<int>& contagem_preto,
7      vector<int>& contagem_branco,
8      int& total_permutacoes_validas)
9  {
10     int tamanho_linha = current_cells.size();
11
12     while (posicao_atual < tamanho_linha &&
13         current_cells[posicao_atual] == -1) {
14         posicao_atual++;

```

```

15     }
16
17     if (indice_bloco == restr.size()) {
18         for (int k = posicao_atual; k < tamanho_linha;
19             ++k)
20         {
21             if (current_cells[k] == 1) {
22                 return;
23             }
24         }
25
26         total_permutacoes_validas++;
27         for (int k = 0; k < tamanho_linha; ++k) {
28             if (current_cells[k] == 1) {
29                 contagem_preto[k]++;
30             } else {
31                 contagem_branco[k]++;
32             }
33         }
34         return;
35     }
36
37     int tamanho_bloco = restr[indice_bloco];
38
39     int tam_blocos_restantes = 0;
40     for (size_t j = indice_bloco + 1; j < restr.size();
41         ++j) {
42         tam_blocos_restantes += restr[j];
43     }
44     int espaco_req_rest = tam_blocos_restantes +
45     (restr.size() - 1 - indice_bloco);
46
47     int max_start = tamanho_linha - espaco_req_rest
48     - tamanho_bloco;
49
50     if (max_start < posicao_atual) {
51         return;
52     }
53
54     for (int start = posicao_atual; start <= max_start;
55         ++start) {
56         if (start > posicao_atual &&
57             current_cells[start - 1] == 1) {
58             break;
59         }
60         if (permutacao_consistente(current_cells, start,
61             tamanho_bloco)) {
62             vector<int> next_cells = current_cells;
63             for (int k = start;
64                 k < start + tamanho_bloco;

```

```

65         ++k) {
66             next_cells[k] = 1;
67         }
68         int pos_separador = start + tamanho_bloco;
69         if (pos_separador < tamanho_linha) {
70             next_cells[pos_separador] = -1;
71         }
72         gerar_permutacao(
73             indice_bloco + 1,
74             pos_separador + 1,
75             next_cells,
76             restr,
77             contagem_preto,
78             contagem_branco,
79             total_permutacoes_validas
80         );
81     }
82 }
83 }

```

Essa função é o cerne da heurística. Ela constrói recursivamente todas as sequências de bloco válidas para uma única linha (ou coluna), contabilizando as células que são resolvidas por certeza lógica. A função começa otimizando o ponto de partida, saltando sobre quaisquer células brancas já marcadas, pois um bloco preto nunca pode começar ali. Logo após, determina o ponto de início mais tardio (max start) possível para o bloco atual, garantindo que o espaço restante seja suficiente para acomodar todos os blocos subsequentes e seus separadores. Se o espaço não for suficiente, a função retorna e poda o ramo. A função então itera por todas as posições de início válidas. Dentro do loop, dois mecanismos de podas são aplicados:

- Poda de cobertura: verifica se o bloco atual está prestes a pular uma célula preta já existente. Se isso ocorrer, a permutação é descartada imediatamente, garantindo que todas as células pretas sejam obrigatoriamente cobertas por algum bloco.
- Uma função auxiliar (permutacao consistente) garante que o bloco não invada células brancas já marcadas na grade.

Para cada posição válida, o estado atual da linha é copiado, o bloco e seu separador são marcados na cópia e a função é chamada recursivamente para o próximo bloco, começando após o separador. Se todos os blocos forem colocados com sucesso e o restante da linha for vazio ou branco, a permutação é considerada válida.

Voltando então para a função "aplicar permutações", a linha 22 é utilizada para checar por inconsistências. Na linha 28 então ocorre a aplicação da superposição global de acordo com as permutações válidas.

A função auxiliar "permutação consistent" é mostrada a seguir:

```

1  bool permutacao_consistente(const vector<int>& cells,
2  int start, int tamanho_bloco) {
3  int tamanho_linha = cells.size();
4
5  for (int k = start; k < start + tamanho_bloco; ++k) {
6      if (cells[k] == -1) {
7          return false;
8      }
9  }
10
11  if (start > 0 && cells[start - 1] == 1) {
12      return false;
13  }
14
15  int pos_separador = start + tamanho_bloco;
16  if (pos_separador < tamanho_linha &&
17  cells[pos_separador] == 1) {
18      return false;
19  }
20
21  return true;
22 }

```

Para além disso, quando a fase heurística é concluída mas o tabuleiro não está completo, o resto é preenchido com uma busca exaustiva utilizando backtracking. O código relacionado ao backtrack não será analisado aqui por não ser o foco, mas pode ser visto no código fonte enviado junto do relatório.

A fase heurística é dominada pelo custo da função "gerar permutacao", que é chamada repetidamente dentro de um loop de estabilização. A complexidade desta fase é altamente dependente da instância do Nonograma.

O custo de gerar permutação é, no pior caso, exponencial em relação ao número de blocos (K) e tamanho da linha (N).

- Para uma linha de tamanho N com K blocos, o número de permutações válidas pode ser modelado pelo número de maneiras de particionar N em K partes. A complexidade é tipicamente $\mathcal{O}(N \cdot 2^K)$ ou $\mathcal{O}(N^K)$, mas é rigidamente limitada pelas células marcadas.
- A complexidade real é $\mathcal{O}(N \cdot P)$, onde P é o número de permutações que o algoritmo gera.
- Se a linha já estiver quase resolvida com muitas células pretas e brancas, a função faz um trabalho mínimo pois a poda ocorre rapidamente, e P é próximo de 1.

O loop de estabilização em "solve nonogram" é correto pois as marcações forçadas em uma linha podem forçar novas marcações em colunas e o loop continua até que não haja mais mudanças, garantindo que a grade atinja um

ponto fixo determinístico, onde não há mais deduções lógicas simples a serem feitas.

O algoritmo foi aplicado a instâncias achadas no site da seguinte referência [4] e comparados com os resultados da referência [5]. A tabela de resultados das comparações estará num README associado a entrega deste trabalho.

5 Conclusão

O desenvolvimento deste solucionador de Nonograma resultou em um algoritmo interessante de se implementar. Uma das partes mais interessantes do desenvolvimento desse projeto foi procurar a literatura para ver como se resolviam nonogramas hoje em dia. Além do mais, tive até mesmo de aprender a resolver bem os nonogramas para entender como funcionava a técnica que implementei. O próximo trabalho que deverá ser implementado com meta-heurísticas pode ser bastante interessante e vai ser legal comparar os resultados com os obtidos nesse trabalho.

6 Referências e bibliografia

References

- [1] K.J. Batenburg and W.A. Kusters. “Solving Nonograms by combining relaxations”. In: *Pattern Recognition* 42.8 (2009). Advances in combinatorial image analysis, pp. 1672–1683. ISSN: 0031-3203. DOI: <https://doi.org/10.1016/j.patcog.2008.12.003>. URL: <https://www.sciencedirect.com/science/article/pii/S0031320308005153>.
- [2] Rafael Castilho and Luan Welter. *Um estudo sobre Nonogramas*. URL: <https://www.inf.ufpr.br/bcc/tcc/2018/2018%20Um%20Estudo%20sobre%20Nonogramas.pdf>.
- [3] Trieu Hung Tran. *MODELING AND SOLVING THE NONOGRAM PUZZLE USING CONSTRAINT PROGRAMMING*. URL: <https://consystlab.unl.edu/Documents/Theses/Hung-UG-thesis.pdf>.
- [4] Jan Wolter. *Sample Paint-by-Number puzzles Download*. URL: <https://webpbn.com/survey/puzzles/>.
- [5] Jan Wolter. *The ‘pbnsolve’ Paint-by-Number Puzzle Solver*. URL: <https://webpbn.com/pbnsolve.html>.
- [6] I.-Chen Wu et al. “An Efficient Approach to Solving Nonograms”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 5.3 (2013), pp. 251–264. DOI: 10.1109/TCIAIG.2013.2251884.
- [7] Mario G. Figueiró Zemor. *The NP-Completeness of NONOGRAM*. URL: <https://pt.scribd.com/document/383817271/Nonogram>.